8. Depth-First Search (DFS)

```
def dfs(graph, node, visited=None, traversal_order=None):
    if visited is None:
        visited = set()
    if traversal_order is None:
        traversal_order = []

    visited.add(node)
    traversal_order.append(node)

    # Process neighbors deeply before backtracking
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, traversal_order)

    return traversal_order

if __name__ == "__main__":
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

    result = dfs(graph, 'A')
    print("DFS Traversal Order starting from node 'A':", " -> ".join(result))
```

main.py Visualize Run ▶

```
1
2  def dfs(graph, node, visited=None,
   traversal_order=None):
3      if visited is None:
4          visited = set()
5      if traversal_order is None:
6          traversal_order = []
7
8      visited.add(node)
9      traversal_order.append(node)
10
11     # Process neighbors deeply before backtracking
12     for neighbor in graph[node]:
13         if neighbor not in visited:
14             dfs(graph, neighbor, visited,
   traversal_order)
15
16     return traversal_order
17
18 if __name__ == "__main__":
19     graph = {
20         'A': ['B', 'C'],
21         'B': ['D', 'E'],
22         'C': ['F']
```

Output

```
DFS Traversal Order starting from node 'A': A -> B -> D -
> E -> F -> C
```